

InstaPay Receipt Register

A Windows desktop application for processing InstaPay receipt images, extracting transaction data with Gemini AI, allowing human review and correction, validating records against approved business lists, generating date-organized Excel reports, and preparing Outlook emails with receipt-image archives.

Primary purpose: Convert receipt photographs into controlled, reviewable, and reportable transaction records while keeping a human approval step before data is saved.

Overview

The InstaPay Receipt Register is designed for operational teams that receive multiple InstaPay receipt photographs and need a consistent process for registration, validation, reporting, and evidence retention. It is suitable for finance, procurement, operations, reimbursement, supplier-payment, and other business workflows where receipt information must be entered accurately and distributed as a daily report.

The operator selects one or more receipt images. Gemini 3.1 Flash Lite extracts the required fields, and the application displays the results beside the original image. The operator compares the values manually, corrects any mistakes, and selects **Save and Send**. Valid records are stored in SQLite, grouped into date-based Excel reports, packaged with receipt images in a ZIP archive, and opened in a classic Outlook draft for final review.

Features

Receipt processing

- Select and process multiple receipt images in one batch.
- Support common image formats including PNG, JPG, JPEG, and BMP.
- Review each receipt through a vertical thumbnail strip.
- Select a thumbnail to display its full receipt preview and extracted fields.
- Keep the image preview contained within the application layout while resizing the window.
- Display processing progress while receipts are extracted sequentially.
- Highlight selected, saved, and error-state thumbnails.

Gemini AI extraction

The application uses the Google GenAI SDK and `gemini-3.1-flash-lite` as the primary model. The extraction prompt requests structured JSON for the following fields:

Field	Extraction rule
Money Value	Numeric transfer amount located next to or before <code>EGP</code> . Values containing commas are normalized for validation.
Account Number/Label	Recipient account number or InstaPay label from the <code>To</code> section only. The sender's account is excluded.
Receipt Number	Reference number shown after the <code>Reference</code> label.
Receipt Date/Time	Date and time shown after the <code>Date</code> label.
Note	Complete text following the <code>Note</code> label.
Note Name	Letter or name portion extracted from the Note section for internal matching.
Note Number	Numeric portion extracted from the Note section for validation and reporting.

The AI output is never treated as automatically approved. The result is displayed for operator review before validation and saving.

Manual review and correction

All extracted values appear in editable fields. The operator can compare the extracted data with the receipt image and manually correct an inaccurate receipt number, account number, date/time, money value, or Note before saving.

The Note field remains a single visible field. Note-name and Note-number values are handled internally to support matching and validation.

Approved accounts

The General Settings section provides a configurable list of approved recipient account numbers or labels. A receipt cannot be saved when its recipient account is not present in this approved list.

Approved Note numbers and names

Settings allows the administrator to create approved Note pairs consisting of:

Approved number	Corresponding name
12345	Example Name

The name is used to help identify the corresponding approved number when the receipt Note contains the name but not the number. If a number is found, the number is used directly and its corresponding name is not required to be present on the receipt.

The Note field also provides live autocomplete suggestions while the user types. Suggestions are based on the approved number/name pairs and do not automatically replace the user's Note text.

Only the approved or resolved **Note number** is written to receipt Excel reports. The corresponding name is used for matching and administration but is not written into the receipt transaction report.

Validation controls

Before saving a batch, the application validates:

- Required receipt number.
- Duplicate receipt numbers already stored in SQLite.
- Duplicate receipt numbers within the current batch.
- Approved recipient account number or label.
- Receipt date and time presence.
- Positive numeric money value.
- Presence of a Note number or a valid name-to-number resolution.
- Approval of the final Note number against the configured approved list.

When an error is found, the first affected receipt is selected automatically and the relevant error is displayed beside the corresponding field.

Local database

SQLite stores the application's operational data locally, including:

- Accepted receipt records.
- Receipt numbers and transaction fields.
- Receipt image paths and raw extraction text.

- Approved account numbers and labels.
- Approved Note number/name pairs.
- Gemini API key setting.
- Outlook recipient list and enabled status.

Receipt numbers are unique in the database to help prevent duplicate registration.

Date-organized Excel reporting

The application uses the date on which the batch is scanned to create the export folder. Within that folder, receipts are separated into Excel files according to their receipt date.

For example, if the batch is scanned on 26-Aug-2026 and contains receipts dated 25-Aug-2026 and 26-Aug-2026:

Plain Text

```
26-Aug-2026/  
├─ Instapay_Receipts_25-Aug-2026.xlsx  
├─ Instapay_Receipts_26-Aug-2026.xlsx  
└─ Instapay_Receipts_Images_26-Aug-2026.zip
```

Receipts sharing the same receipt date are written into the same Excel file. Each receipt-date workbook includes the receipt date as a **Photo Date** field and records only the approved Note number in the Note-related report column.

Receipt-image ZIP archive

One ZIP archive is generated for the current scan batch. It contains all receipt images scanned during that batch, including images whose receipt dates are older than the scan date.

The ZIP follows this naming convention:

Plain Text

```
Instapay_Receipts_Images_<scan-date>.zip
```

Outlook integration

The application uses the installed classic Outlook desktop client through Windows automation. It creates an email draft addressed to the enabled recipients and attaches:

1. Every date-specific receipt Excel report generated for the batch.

2. The single receipt-image ZIP archive for the batch.

The application uses Outlook `.Display()` so the user can review the draft before sending it. The application does not store an Outlook password.

Settings navigation

The Settings window contains separate navigation areas:

- **General:** Outlook recipients, approved accounts, and approved Note number/name pairs.
- **Admin:** Gemini API key and destructive data-management controls.

The Gemini API key is stored using the existing application setting and is accessible only through the password-protected Admin section.

Password-protected Admin controls

The Admin section requires the password `123456` before it opens. It provides a calendar-based date selector and two destructive actions:

Delete Receipt Data

For the selected date, the application locates the export folder named after that date, deletes all receipt Excel files inside it, and removes the receipt records associated with that scan/export date from SQLite. Receipt photos and ZIP archives are retained for the separate photo cleanup action.

Delete Receipt Photos

For the selected date, the application locates the export folder named after that date, deletes its receipt-image ZIP archive, deletes the physical image files associated with receipts scanned on that date, and clears their stored image paths from SQLite while retaining the transaction records.

Both actions require a confirmation dialog before proceeding. Empty export folders are removed after cleanup, while folders containing remaining files are preserved.

Windows deployment preparation

The repository includes:

- PyInstaller specification file.
- Windows build scripts.
- Inno Setup installer script.

- Application icon and contact icons.
- Runtime asset bundling configuration.

The intended production workflow is to build a Windows executable and installer so end users do not need to install Python or the development dependencies manually.

Contact Developer window

The sidebar includes a **Contact Us** button that opens a **Contact Developer** window with clickable WhatsApp, Facebook, and LinkedIn buttons. The buttons include supplied icons, normal gray text, tooltips, hover feedback, and fade animations.

User workflow

1. Open the application.
2. Select one or more receipt images.
3. Select **Extract Data**.
4. Wait for the extraction progress dialog to complete.
5. Select each thumbnail and compare the displayed data with the original receipt.
6. Correct any inaccurate values manually.
7. Review the Note field and use the approved number/name autocomplete when needed.
8. Select **Save and Send**.
9. Correct any validation errors identified by the application.
10. After successful saving, review the generated Excel reports and receipt-image ZIP archive.
11. Review the Outlook draft and send it according to company policy.

Project structure

Plain Text

```
instapay_app/  
├─ app.py           # PySide6 application and user interface  
├─ database.py      # SQLite schema and data access  
├─ services.py      # Gemini extraction and Outlook integrati  
├─ reporting.py     # Validation, Excel reports, and ZIP arch.  
├─ requirements.txt # Python dependencies  
├─ instapay_receipt_register.spec # PyInstaller build configuration  
├─ installer.iss    # Inno Setup installer script
```

```
|— build_windows.ps1      # Windows build helper
|— build_release.bat      # Release build helper
|— ic_edita_logo.png      # Application logo
|— ic_edita_logo.ico      # Windows application icon
|— ic_whatsapp.png        # Contact icon
|— ic_facebook.png        # Contact icon
|— ic_linkedin.png        # Contact icon
```

Requirements

Running from source

- Windows 10 or newer is recommended.
- Python 3.10 or newer.
- A Google Gemini API key.
- Classic Outlook desktop installed and signed in if email distribution is required.
- Internet connectivity when calling Gemini.

Install the Python dependencies from a Windows PowerShell or Command Prompt:

Plain Text

```
python -m venv .venv
.\.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Run the application with:

Plain Text

```
python app.py
```

Dependencies

The main dependencies are:

Package	Purpose
PyQt6	Windows desktop user interface
google-genai	Gemini image and text extraction

Pillow	Image loading and preview support
openpyxl	Excel report generation
pywin32	Outlook desktop automation on Windows
pytest and pytest-qt	Testing support

Configuration

1. Open **Settings** from the left sidebar.
2. Open the **Admin** section and enter the administrator password.
3. Enter the Gemini API key in the Admin section and save the settings.
4. Return to General Settings and add approved recipient accounts or labels.
5. Add every approved Note number together with its corresponding name.
6. Add Outlook recipients and select which recipients should receive reports.
7. Save the settings.

The Gemini API key should be treated as a secret. Do not commit it to the repository, place it in source code, or share it through email or public issue trackers.

Building the Windows application

The repository contains a PyInstaller specification and Windows build helpers. A typical source build is:

Plain Text

```
.\.venv\Scripts\activate
pip install -r requirements.txt
pyinstaller instapay_receipt_register.spec
```

The generated executable should be tested on a clean Windows workstation with the required classic Outlook environment before company distribution. Follow `BUILD_WINDOWS.md` for project-specific build instructions when available.

Data and privacy

Receipt images and transaction data may contain sensitive financial or business information. The local application data directory should be protected by Windows access

controls and included in the company's backup and retention policy.

Receipt images are sent to the configured Gemini service for extraction. Before production deployment, the company should review its information-security requirements, AI vendor terms, data-retention policy, and restrictions on transmitting financial documents to external services.

The application's local database and generated reports should be backed up according to the organization's internal-control requirements. Administrators should consider creating a backup before using the destructive Admin deletion actions.

Production considerations for large organizations

For a large company, the current application is best introduced through a controlled pilot using representative receipts and a defined group of operators. IT and information-security teams should verify Gemini API usage, Outlook automation policies, Windows installer behavior, data-folder permissions, and backup procedures.

Possible future enterprise enhancements include centralized multi-user storage, role-based authentication, an audit trail of edits and deletions, encrypted backups, centralized configuration, ERP integration, dashboard reporting, digital signing, and automated monitoring of extraction failures.

Limitations

AI extraction quality depends on image clarity, receipt layout, language, lighting, and the consistency of the InstaPay design. Human review remains necessary before saving. Outlook automation also depends on the installed classic Outlook client and local company security policies.

The current Admin password is a basic application-level access control. It should be replaced with stronger authentication or protected configuration before a high-security enterprise deployment.

License

No open-source license has been selected yet. Until a license is added to this repository, all rights are reserved by the copyright holder. Add a `LICENSE` file before public redistribution if the project will be shared under a specific license.

Copyright

Plain Text

